



LIBRARY BORROWING AND BOOK TRACKING SYSTEM

Team 04

Lecturer: Mr. ROEUN Pacharoth

I4-GIC-A

Team members & Roles



Neth Tithyarita

e20221538

Security

- Authentication
 - register
 - login
 - logout
- Admin/User Dashboard
 - user management
 - view history



Doeung Sreypy

e20220337

Database

- Work on database
- Search button
- User profile
 - Edit user
 - Change password



Kaing Sophea

e20220329

Main CRUD

- Book management
 - Add Book
 - Update Book
 - Delete Book



Meng Leangei

e20221618

Secondary CRUD

- Borrow management
 - Booking
 - Confirm Borrow
 - Confirm Return
 - BorrowRecord



Guov Houysang

e20220816

Frontend

- Work on front-end
 - Design/ Edit all webpages
 - Tester

TABLE CONTENTS

➤	Introduction	➤	Technology use
➤	Project Planning	➤	Strength & Weakness
➤	Database Design	➤	Challenge
➤	UML Design	➤	Conclusion
➤	Design workflow	➤	Demo



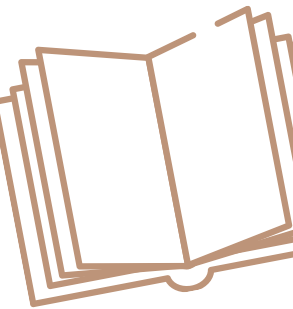
INTRODUCTION

The Library and Borrow Book Tracking System is a computerized solution designed to manage library books and borrowing activities efficiently. It replaces traditional manual record-keeping with a digital database that allows librarians to track available, borrowed, and returned books accurately. The system helps reduce errors, save time, and improve overall library efficiency by supporting book management, user management, and borrow/return tracking. It is designed for small to medium-sized libraries such as school or college libraries.



PROJECT PLANNING

- Define project requirement
- Divided Task
- Design Database
- Design UML
- Set up project
- Coding, Testing, fixing step by step



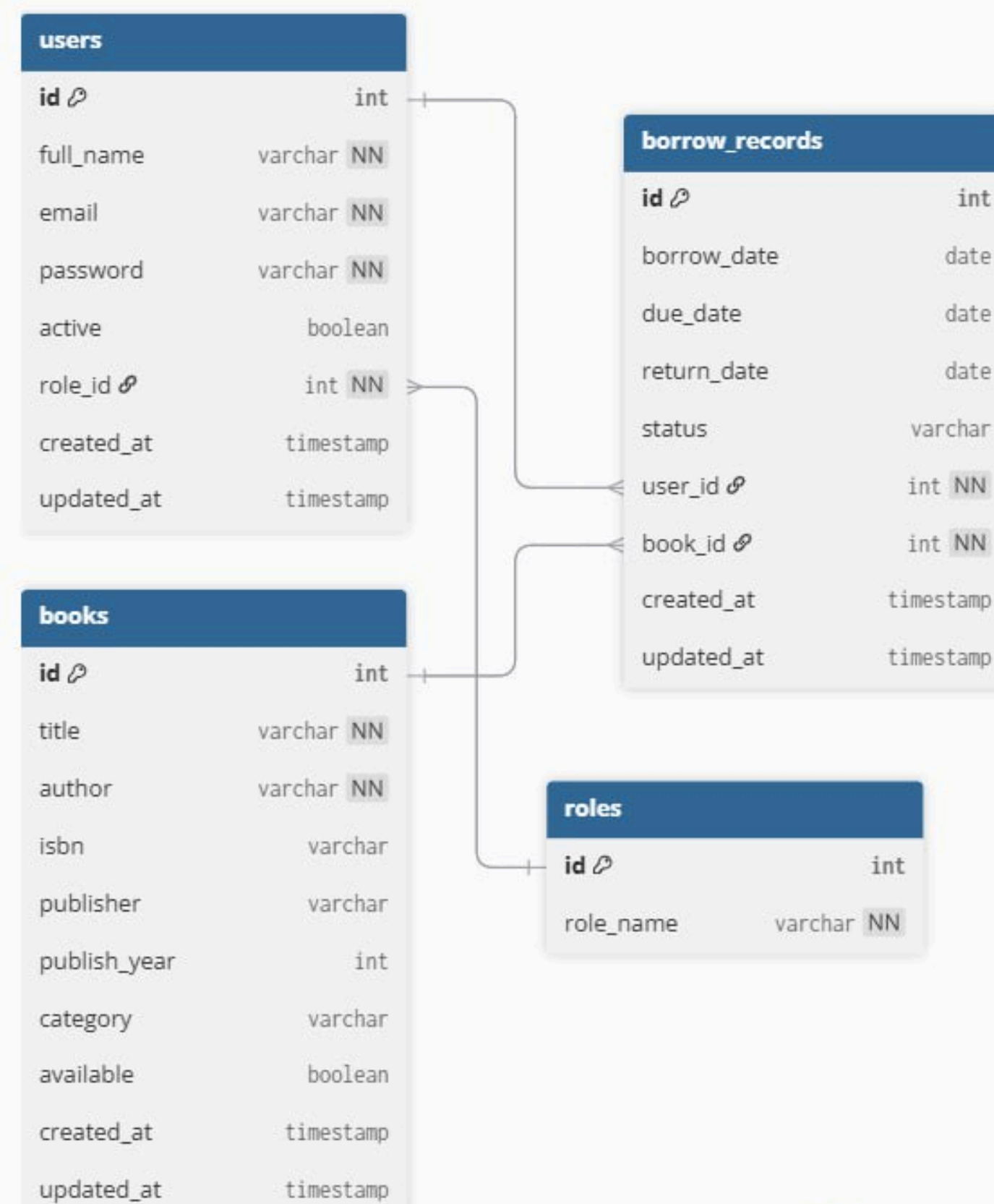
WEEKLY PROGRESS

Week	Progress of the project
week 1	Set up GitHub, divide the role for members, draw database
week 2	Authenticstion(register, login), create the entity table and entity of books, borrow record, user, borrow, role in database and front-end
week 3	Fixed errors after merging code, implemented book selection and borrowing, Managed borrow list, Added login/logout functionality
week 4	Designed register page, completed user management (add, edit, view, history), Managed books (list, add, edit, delete), Handled borrow/return records, Updated user account features (view/edit info, change password), Designed front-end pages (welcome, home, account, profile, headers for librarian)



DATABASE DESIGN

- **4 main entities:** Users, Roles, Books, Borrow_Records
- **Normalized to 3NF:** Eliminates redundancy and duplicate data
- Relationships handled by foreign keys:
 - Users → Roles
 - Users → Borrow_Records
 - Books → Borrow_Records
 - Borrow_Records acts as a bridge between Users and Books



UML DESIGN

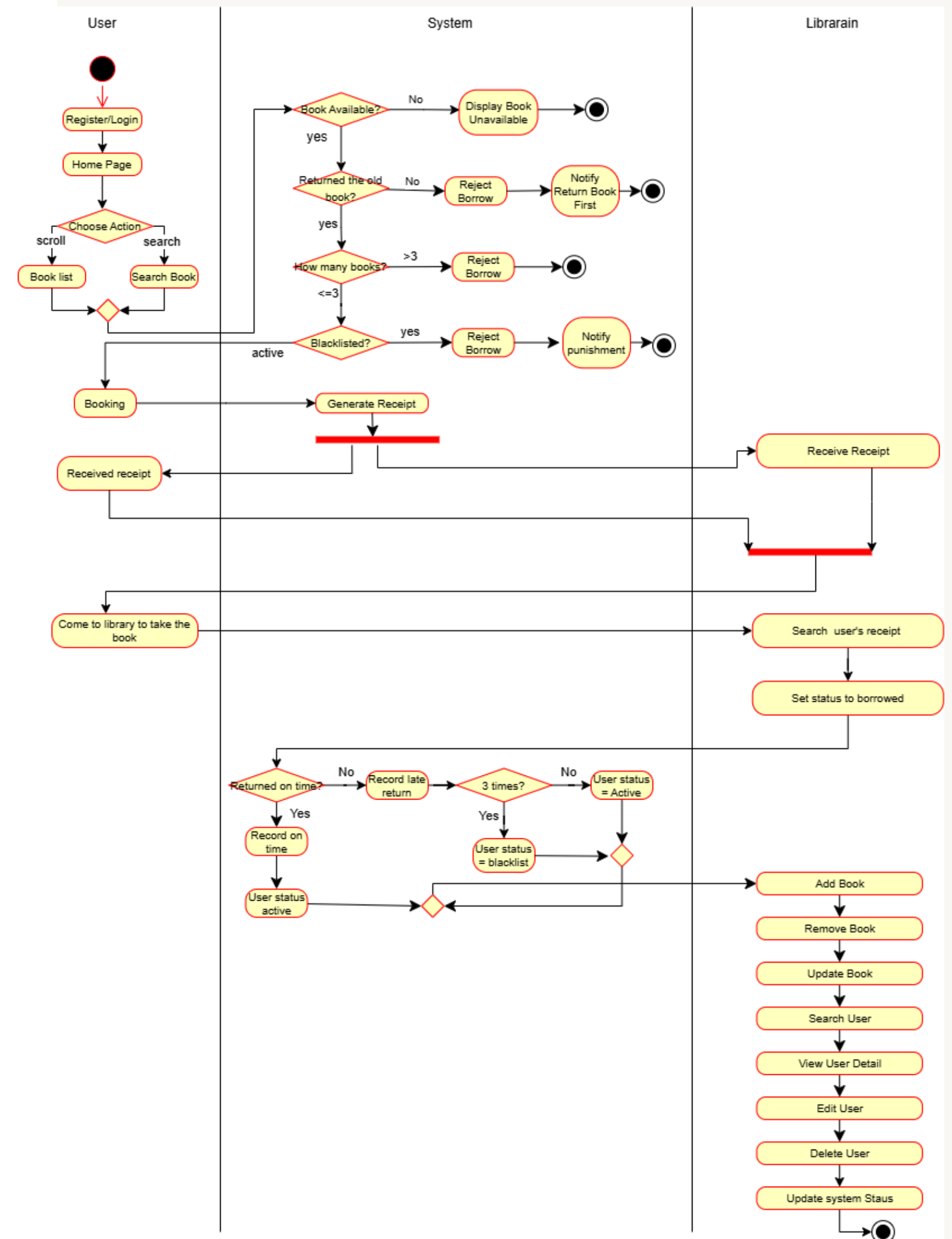


Diagram type: Activity Diagram



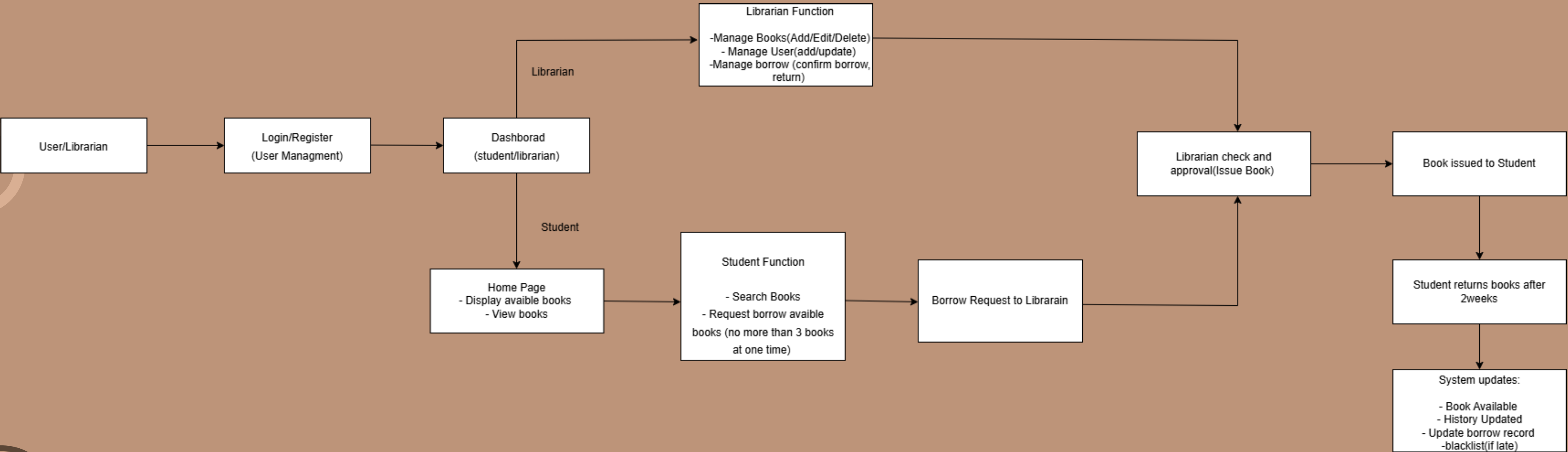
Actor: User | System | Librarian

- **User** → Search books, request booking, receive receipt, return books
- **System** → Validate rules, generate receipts and track borrow status
- **Librarian** → Verify receipt, confirm borrowing, manage books and users

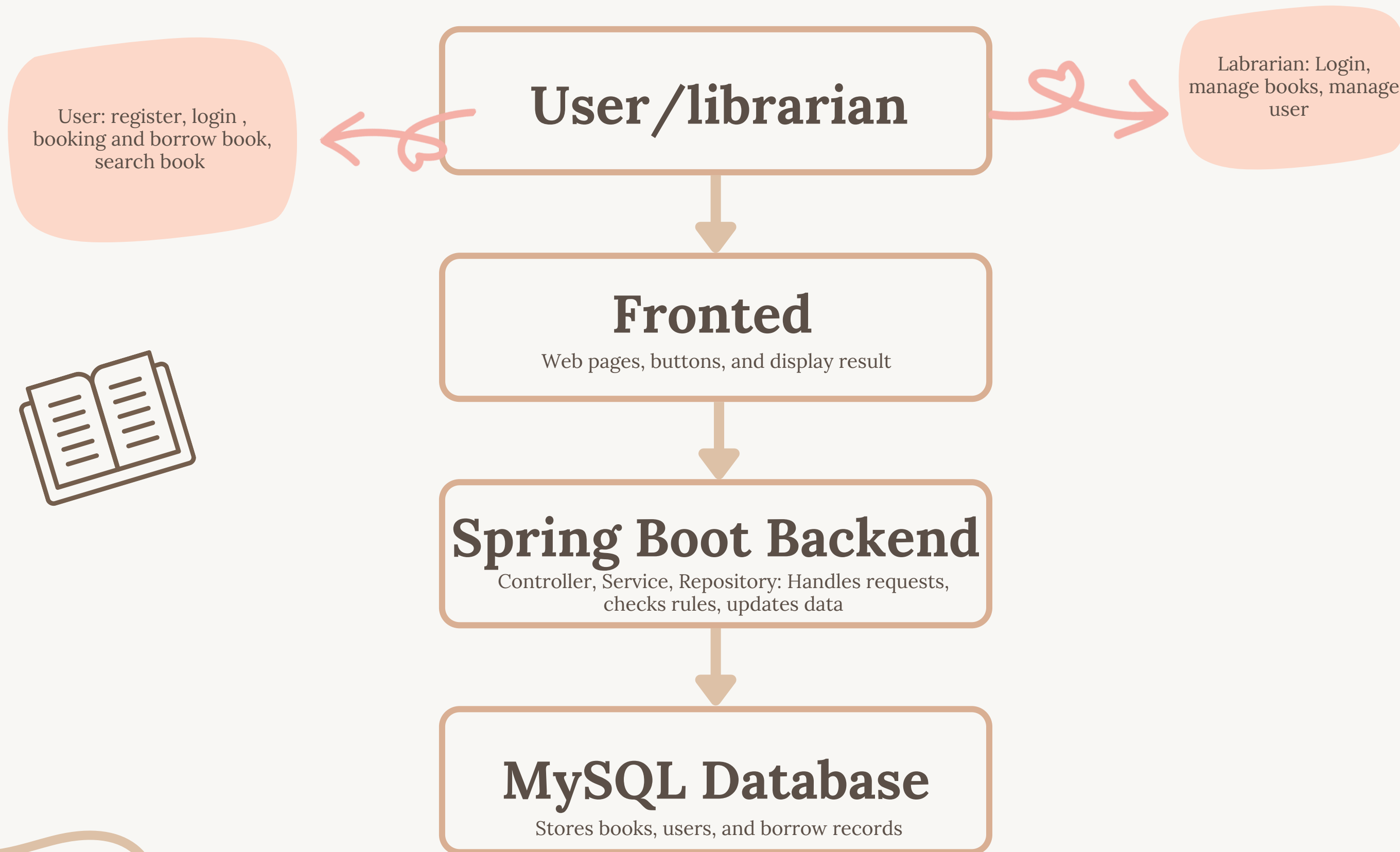




DATA FLOW

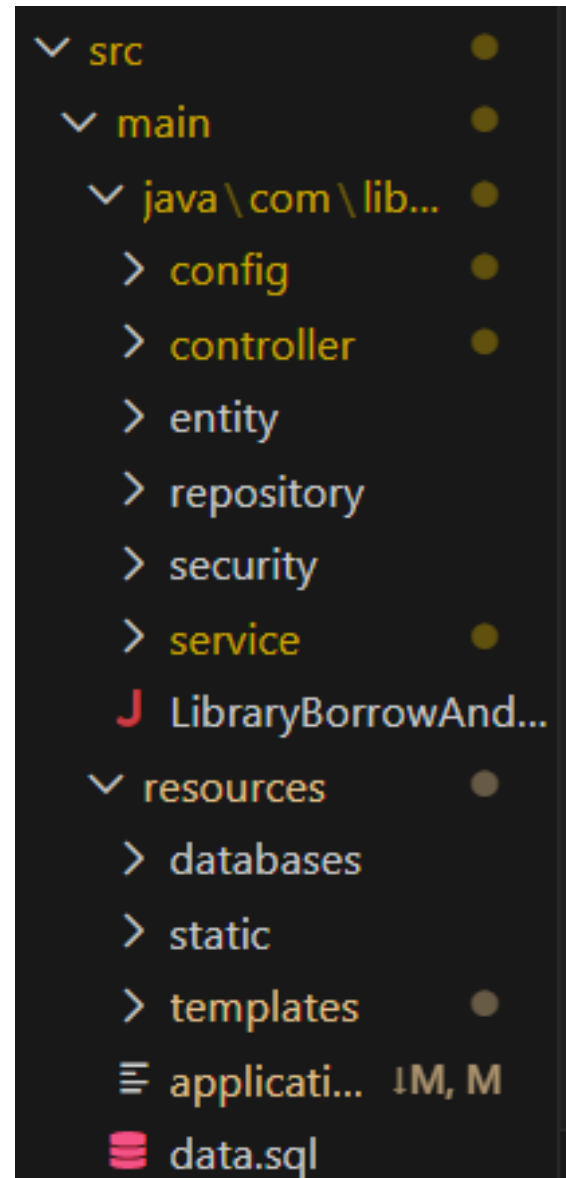


DESIGN WORKFLOW

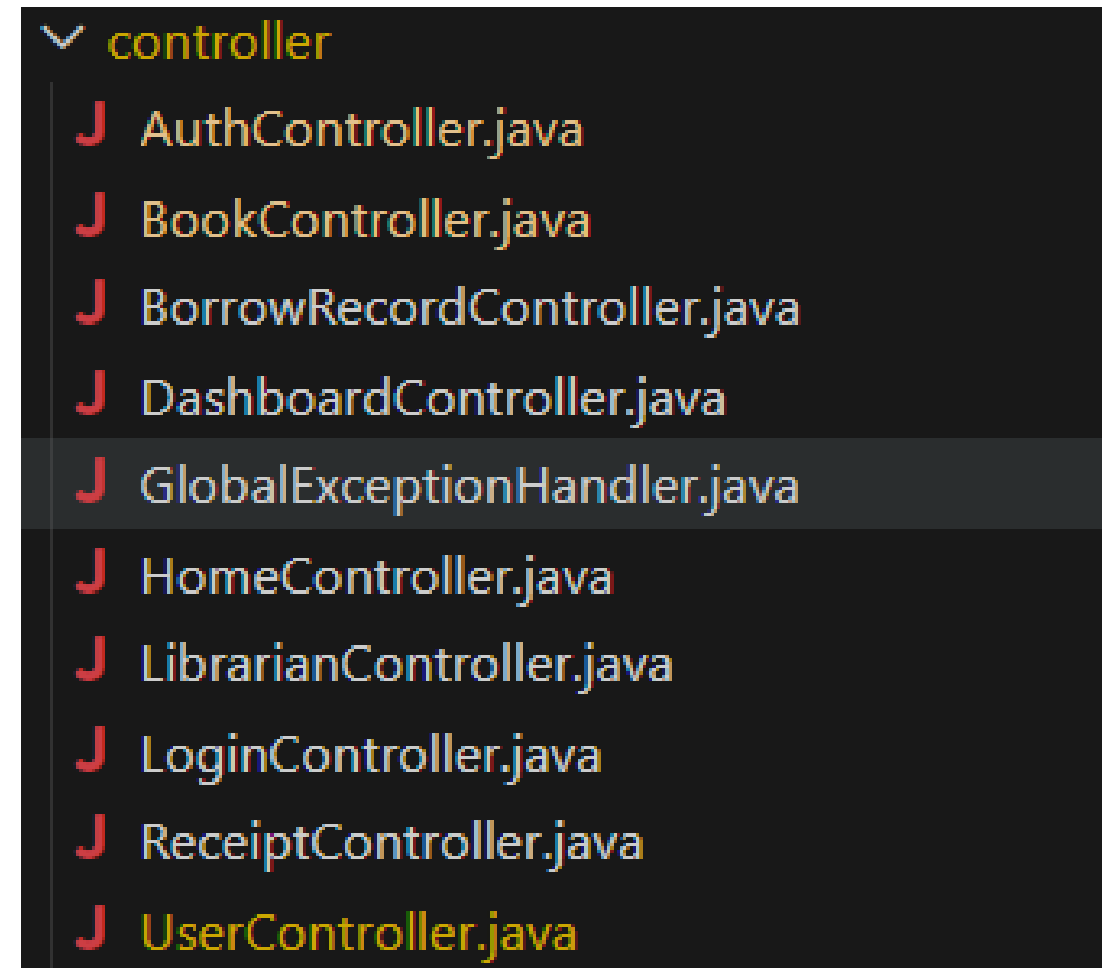


Folder

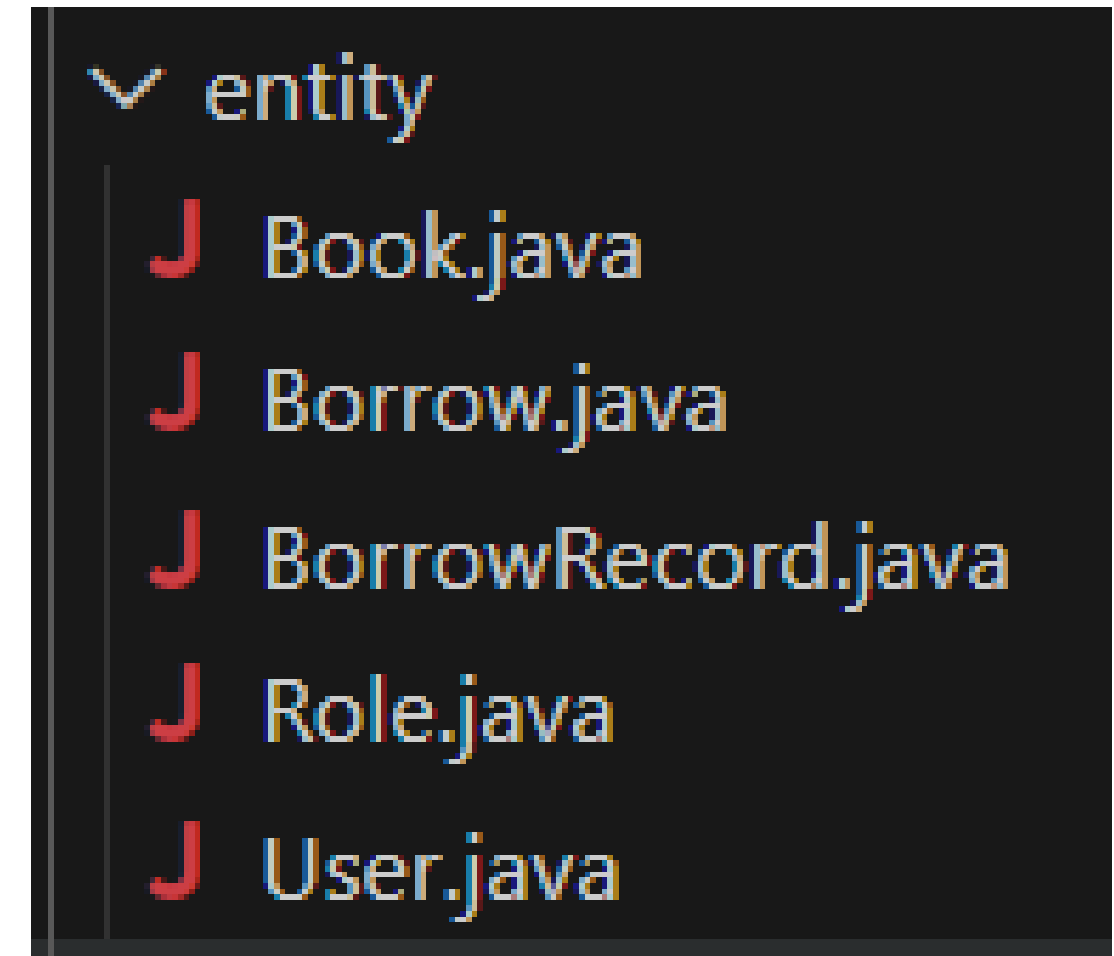
Folder Structure



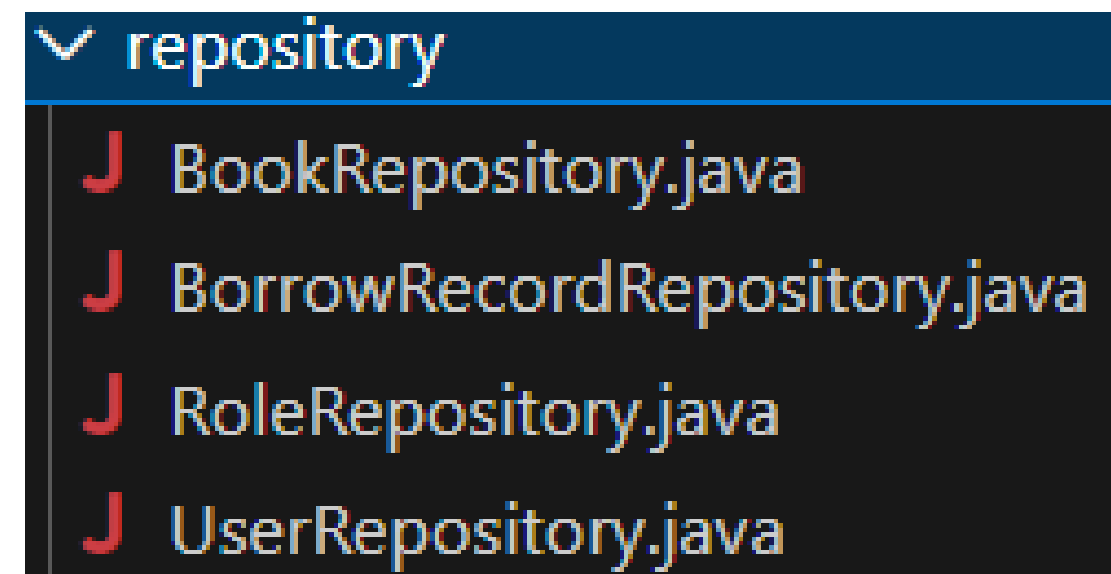
Controller



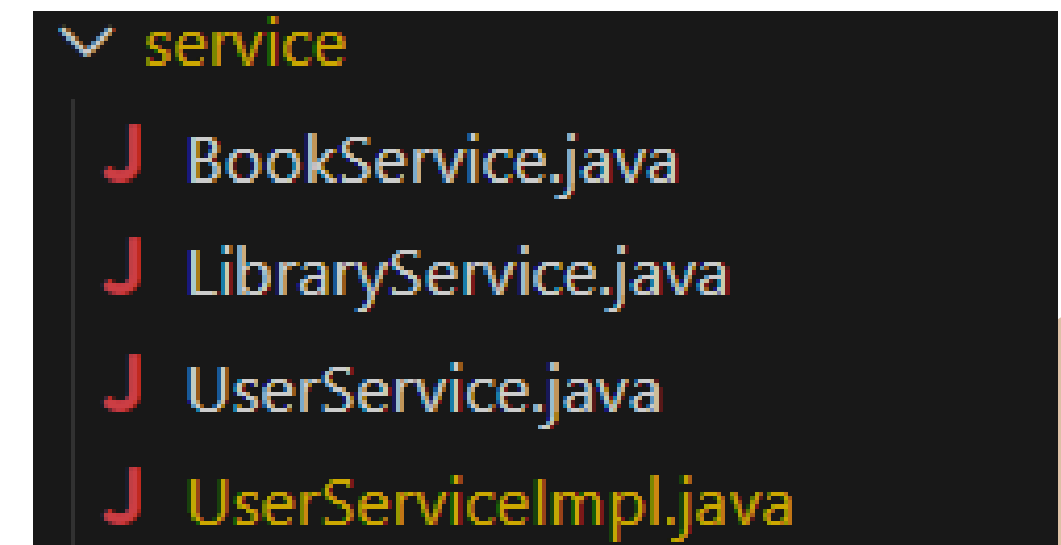
Entity



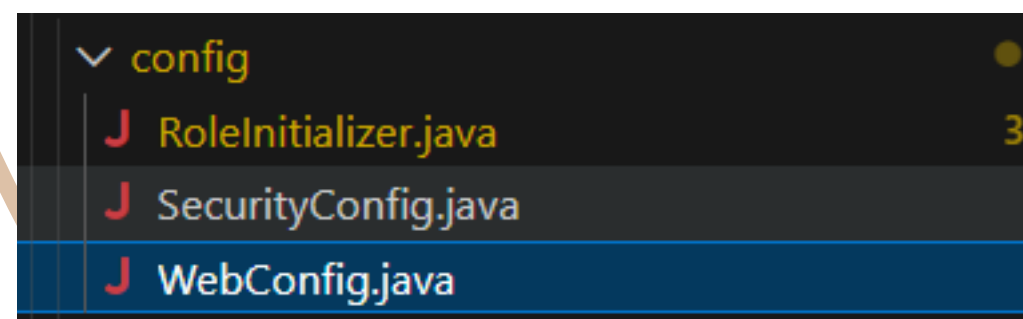
Repository



Service

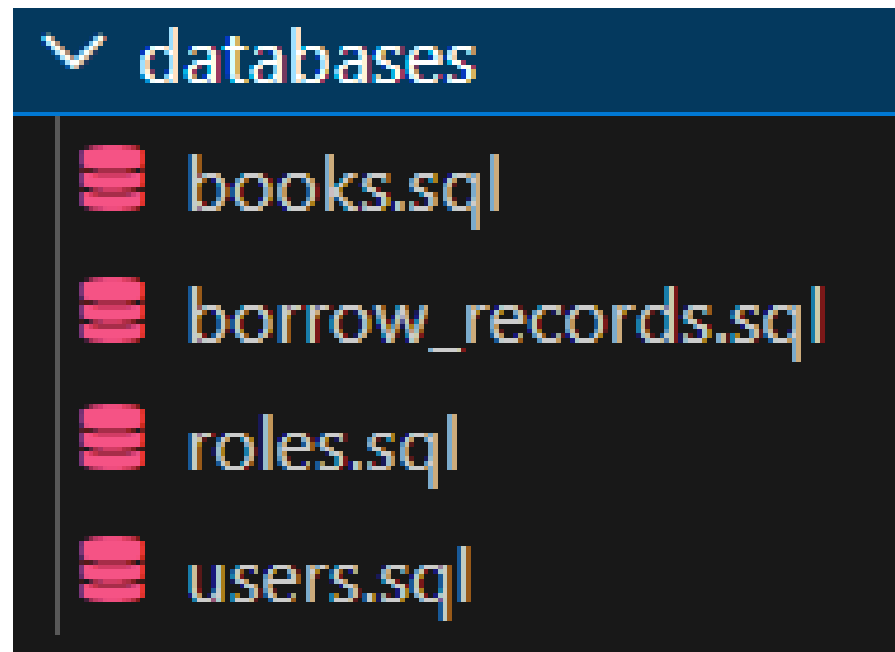


Config

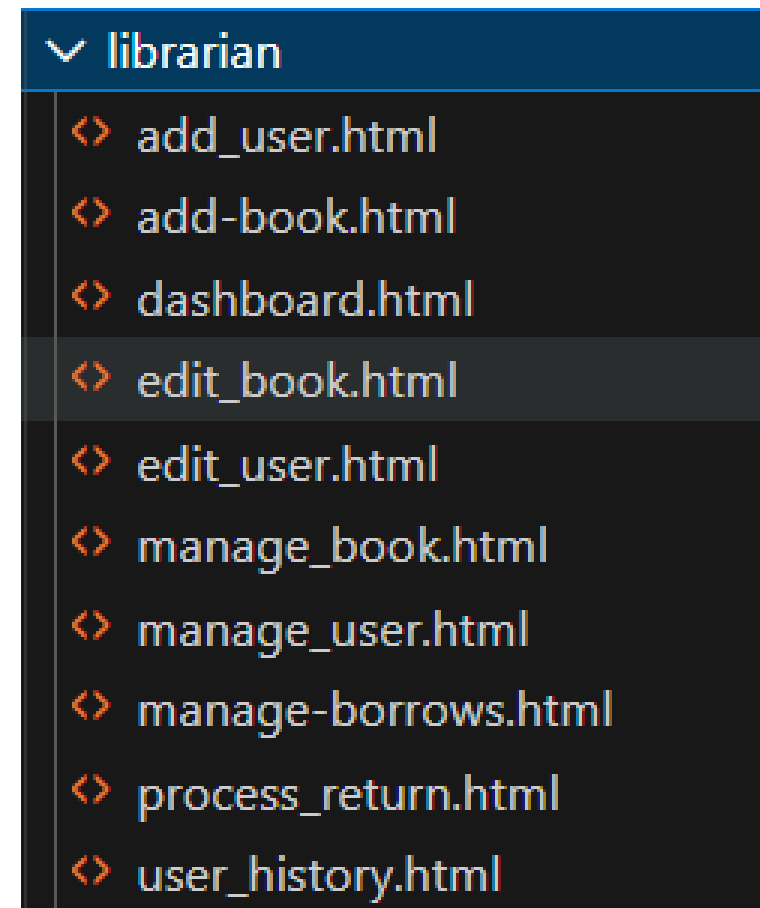


Folder

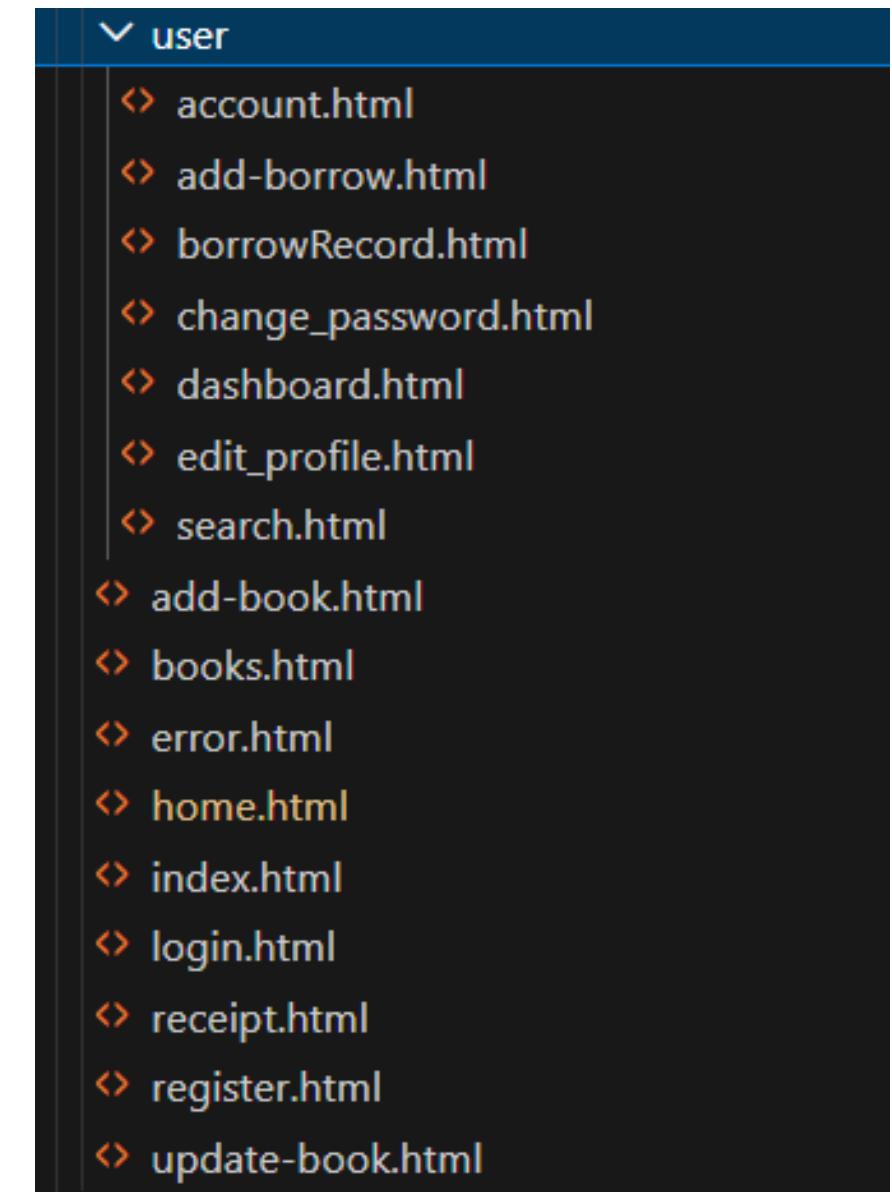
Database tables



Librarian html



user html



Authentication APIs

HTTP Method	Endpoint	Description
GET	/api/auth/register	Display user registration page
POST	/api/auth/register	Register a new user (save user to database)

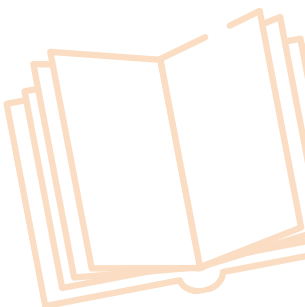


librarian Management

HTTP Method	Endpoint	Description	View / Result
GET	/librarian/manage_user	View & search users	librarian/manage_user.html
GET	/librarian/add-user	Show add user form	librarian/add_user.html
POST	/librarian/add-user	Add new user	Redirect /librarian/manage_user
GET	/librarian/edit-user/{id}	Show edit user form	librarian/edit_user.html
POST	/librarian/edit-user/{id}	Update user	Redirect /librarian/manage_user
POST	/librarian/manage_user/deactivate/{id}	Deactivate user	Redirect /librarian/manage_user
POST	/librarian/manage_user/reactivate/{id}	Reactivate user	Redirect /librarian/manage_user
GET	/librarian/user/{id}/history	View user borrow history	librarian/user_history.html

User Management

HTTP Method	Endpoint	Description	View / Result
GET	/user/account	View user account	user/account.html
GET	/user/account/edit	Edit profile form	user/edit_profile.html
POST	/user/account/edit	Update profile	Redirect /user/account
GET	/user/change_password	Change password form	user/change_password.html
POST	/user/account/change-password	Update password	Redirect /user/account



AUTHENTICATION

Controller

```
@GetMapping("/register")
public String showRegisterPage(Model model) {
    model.addAttribute(attributeName: "user", new User());
    return "register";
}

@PostMapping("/register")
public String register(@ModelAttribute("user") User user, Model model) {
    // Check email uniqueness
    if (userRepository.findByEmail(user.getEmail()).isPresent()) {
        model.addAttribute(attributeName: "error", attributeValue: "Email already exists!");
        return "register";
    }

    // Encode password
    user.setPassword(passwordEncoder.encode(user.getPassword()));

    // Set default values
    user.setActive(active: true);
    LocalDateTime now = LocalDateTime.now();
    user.setCreatedAt(now);
    user.setUpdatedAt(now);

    // Assign default USER role
    Optional<Role> roleOptional = roleRepository.findByRoleName(roleName: "USER");
    Role role;
    if (roleOptional.isPresent()) {
        role = roleOptional.get();
    } else {
        // Create USER role if it doesn't exist
        role = new Role();
        role.setRoleName(roleName: "USER");
        role = roleRepository.save(role);
    }
    user.setRole(role);

    // Save user to database
    userRepository.save(user);
    You, 3 weeks ago • combine code
    return "redirect:/login";
}
```

Service

```
@Override
public UserDetails loadUserByUsername(String email) throws UsernameNotFoundException {
    User user = userRepository.findByEmail(email)
        .orElseThrow(() -> new UsernameNotFoundException(msg: "User not found"));

    String role = (user.getRole() != null && user.getRole().getRoleName() != null)
        ? user.getRole().getRoleName()
        : "USER"; // fallback role

    boolean active = user.getActive() != null ? user.getActive() : true; // Unboxing possible

    return org.springframework.security.core.userdetails.User.builder()
        .username(user.getEmail())
        .password(user.getPassword())
        .roles(role)
        .disabled(!active) // Houysang, last week • Fix the last front-end ...
        .build();
}
```

Repository

```
public interface UserRepository extends JpaRepository<User, Long> {

    Optional<User> findByEmail(String email);

    // ✅ Add this to enable searching users by full name or email
    List<User> findByFullNameContainingIgnoreCaseOrEmailContainingIgnoreCase(String fullName, String email);
}
```

Book Management APIs

HTTP Method	Endpoint	Description	View Returned
GET	/books	Display list of all books	books.html
GET	/books/add	Show add book form	add-book.html
POST	/books/save	Save a new book	Redirect to /books
GET	/books/edit/{id}	Show edit book form	update-book.html
POST	/books/edit/{id}	Update book information	Redirect to /books
POST	/books/delete/{id}	Delete a book	Redirect to /books



Book Management

Controller

```
@Controller
@RequestMapping("/books")
public class BookController {

    private final BookService bookService;

    public BookController(BookService bookService) {
        this.bookService = bookService;
    }

    @GetMapping
    public String listBooks(Model model) {
        model.addAttribute(attributeName: "books", bookService.getAllBooks());
        return "books";
    }

    @GetMapping("/add")
    public String showAddForm(Model model) {
        model.addAttribute(attributeName: "book", new Book());
        return "add-book";
    }

    @PostMapping("/save")
    public String saveBook(@ModelAttribute("book") Book book,
        @RequestParam(value = "coverFile", required = false) MultipartFile coverFile) throws IOException {

        if (coverFile != null && !coverFile.isEmpty()) {
            String filename = System.currentTimeMillis() + "_" + coverFile.getOriginalFilename();
            Path uploadPath = Paths.get(System.getProperty(key: "user.dir"), ...more: "uploads", "books");
            Files.createDirectories(uploadPath);
            Path filePath = uploadPath.resolve(filename);
            coverFile.transferTo(filePath.toFile());    Null type safety: The expression of type 'File' needs unchecked convers
            book.setCoverUrl("/uploads/books/" + filename);
        }

        bookService.saveBook(book);
        return "redirect:/books";
    }

    @GetMapping("/edit/{id}")
    public String showEditForm(@PathVariable("id") Long id, Model model) {
        Book book = bookService.getBookById(id)
            .orElseThrow(() -> new IllegalArgumentException("Invalid book Id: " + id));
        model.addAttribute(attributeName: "book", book);
        return "update-book";
    }

    @PostMapping("/edit/{id}")
    public String updateBook(@PathVariable("id") Long id,
        @ModelAttribute("book") Book formBook,
        @RequestParam(value = "coverFile", required = false) MultipartFile coverFile) throws IOException {

        // Load the existing book from DB
        Book existingBook = bookService.getBookById(id)
            .orElseThrow(() -> new IllegalArgumentException("Invalid book Id: " + id));

        // Update all fields except cover
        existingBook.setTitle(formBook.getTitle());
        existingBook.setAuthor(formBook.getAuthor());
        existingBook.setIsbn(formBook.getIsbn());
        existingBook.setPublisher(formBook.getPublisher());
        existingBook.setPublishYear(formBook.getPublishYear());
        existingBook.setCategory(formBook.getCategory());
        existingBook.setQuantity(formBook.getQuantity());
        existingBook.setAvailable(formBook.getQuantity() != null && formBook.getQuantity() > 0);

        // Update cover only if a new file is uploaded
        if (coverFile != null && !coverFile.isEmpty()) {
            String filename = System.currentTimeMillis() + "_" + coverFile.getOriginalFilename();
            Path uploadPath = Paths.get(System.getProperty(key: "user.dir"), ...more: "uploads", "books");
            Files.createDirectories(uploadPath);
            Path filePath = uploadPath.resolve(filename);
            coverFile.transferTo(filePath.toFile());    Null type safety: The expression of type 'File' needs unchecked convers
            existingBook.setCoverUrl("/uploads/books/" + filename);
        }

        // Save the updated book
        bookService.saveBook(existingBook);

        return "redirect:/books";
    }
}
```

Service

```
@Service
public class BookService {

    private final BookRepository bookRepository;

    public BookService(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
    }

    // LIST ALL BOOKS
    public List<Book> getAllBooks() {
        return bookRepository.findAll();
    }

    // SAVE NEW BOOK
    public void saveBook(Book book) {
        book.setCreatedAt(LocalDateTime.now());
        book.setUpdatedAt(LocalDateTime.now());
        bookRepository.save(book);
    }

    // GET BOOK BY ID
    public Optional<Book> getBookById(Long id) {
        return bookRepository.findById(id);    Null type safety: The expression of type 'Long' needs unchecked conversion
    }

    // UPDATE BOOK
    public void updateBook(Long id, Book updatedBook) {
        Book existingBook = bookRepository.findById(id)    Null type safety: The expression of type 'Long' needs unchecked conversion
            .orElseThrow(() -> new IllegalArgumentException("Invalid book Id: " + id));

        existingBook.setTitle(updatedBook.getTitle());
        existingBook.setAuthor(updatedBook.getAuthor());
        existingBook.setIsbn(updatedBook.getIsbn());
        existingBook.setPublisher(updatedBook.getPublisher());
        existingBook.setPublishYear(updatedBook.getPublishYear());
        existingBook.setCategory(updatedBook.getCategory());
        existingBook.setQuantity(updatedBook.getQuantity());
        existingBook.setAvailable(updatedBook.getAvailable());
        existingBook.setCoverUrl(updatedBook.getCoverUrl());
        existingBook.setUpdatedAt(LocalDateTime.now());

        bookRepository.save(existingBook);
    }

    // DELETE BOOK
    public void deleteBook(Long id) {
        bookRepository.deleteById(id);    Null type safety: The expression of type 'Long' needs unchecked conversion
    }
}
```

Repository

```
public interface BookRepository extends JpaRepository<Book, Long> {

    // Get latest 8 books for featured section
    List<Book> findAllOrderByCreatedAtDesc();
    // Search books by title, author, or category
    List<Book> findByTitleContainingIgnoreCaseOrAuthorContainingIgnoreCaseOrCategoryContainingIgnoreCase(
        String title,
        String author,
        String category
    );
    // Find by category
    List<Book> findByCategoryIgnoreCase(String category);
    // Find by category and search query
    List<Book> findByCategoryIgnoreCaseAndTitleContainingIgnoreCaseOrAuthorContainingIgnoreCase(String category, String title);
}
```

Borrow Management

HTTP Method	Endpoint	Description	View / Result
GET	/user/booking/{bookId}	Reserve a book by selecting it	Redirect /user/home
GET	/user/borrowRecord/{bookId}	Select book from search	redirect:/user/dashboard
POST	/user/borrowRecord/add	Create borrow booking	Redirect to /user/dashboard
GET	/user/borrowRecord	View all borrow records	user/borrowRecord.html
POST	/user/borrowRecord/{id}/delete	Delete borrow record	Redirect to /user/borrowRecord
GET	/librarian/records	View pending borrow records	librarian/manage-borrows.html
POST	/librarian/borrow/confirm/{id}	Confirm borrow	Redirect /librarian/records
GET	/librarian/process_return	View borrowed books (return)	librarian/process_return.html
POST	/librarian/borrow/return/{id}	Confirm book return	Redirect /librarian/process_return
POST	/librarian/borrow/delete/{id}	Delete borrow record	Redirect /librarian/process_return
GET	/librarian/records/search	Search borrow records	librarian/process_return.html
GET	/librarian/user/{id}/history	View user borrow history	View pending borrow records

Borrow Management

Controller

Book Reservation

```
@GetMapping("/user/booking/{bookId}")
public String book(@PathVariable("bookId") Long bookId, RedirectAttributes redirect, Principal principal) {
    try {
        libraryService.bookReservation(bookId, principal.getName());
        redirect.addFlashAttribute(attributeName: "message", attributeValue: "Booking created. Please confirm at the library.");
        return "redirect:/user/receipt"; // handled by ReceiptController
    } catch (RuntimeException e) {
        redirect.addFlashAttribute(attributeName: "error", e.getMessage());
        return "redirect:/user/home";
    }
}

@PostMapping("/user/borrowRecord/add")
public String createBooking(
    @RequestParam("bookId") Long bookId,
    RedirectAttributes redirectAttributes, Principal principal) {

    try {
        libraryService.bookReservation(bookId, principal.getName());

        redirectAttributes.addFlashAttribute(
            attributeName: "successMessage",
            attributeValue: "Book booked successfully. Waiting for confirmation."
        );

        return "redirect:/user/dashboard";
    } catch (Exception e) {
        redirectAttributes.addFlashAttribute(
            attributeName: "errorMessage",
            e.getMessage()
        );
    }
    return "redirect:/user/home";
}
```

userController

Service

```
@Transactional
private int getActiveBorrowCount(Long userId) {
    long booked = borrowRecordRepository.countByUserIdAndStatus(userId, status: "BOOKED");
    long borrowed = borrowRecordRepository.countByUserIdAndStatus(userId, status: "BORROWED");
    return (int) (booked + borrowed);
}

public BorrowRecord bookReservation(Long bookId, String email) {
    User user = userRepository.findById(email)
        .orElseThrow(() -> new RuntimeException(message: "User not found"));

    if (getActiveBorrowCount(user.getId()) >= MAX_BORROW_LIMIT) {
        throw new RuntimeException(message: "You can only borrow up to 3 books at one time");
    }

    Book book = bookRepository.findById(bookId)
        .orElseThrow(() -> new RuntimeException(message: "Book not found"));

    if (!book.getAvailable()) {
        throw new RuntimeException(message: "Book is not available");
    }

    BorrowRecord record = new BorrowRecord();
    record.setUser(user);
    record.setBook(book);
    record.setIsbn(book.getIsbn());
    record.setStatus(status: "BOOKED");
    record.setCreatedAt(LocalDateTime.now());

    // decrement quantity for reservation
    Integer currentQty = book.getQuantity() == null ? 0 : book.getQuantity();
    book.setQuantity(currentQty - 1);
    book.setAvailable(book.getQuantity() != null && book.getQuantity() > 0);
    bookRepository.save(book);

    return borrowRecordRepository.save(record);
}
```

Controller

Confirm Borrow

```
@PostMapping("/borrow/confirm/{id}")
public String confirmBorrow(@PathVariable("id") Long id, RedirectAttributes redirect) {
    try {
        libraryService.confirmBorrow(id);
        redirect.addFlashAttribute(attributeName: "success", attributeValue: "Borrow confirmed successfully");
    } catch (RuntimeException e) {
        redirect.addFlashAttribute(attributeName: "error", e.getMessage());
    }
    return "redirect:/librarian/records";
}
```

libraryController

Service

```
@Transactional
public void confirmBorrow(Long recordId) {
    BorrowRecord record = borrowRecordRepository.findById(recordId)
        .orElseThrow(() -> new RuntimeException(message: "Borrow record not found"));

    record.setStatus(status: "BORROWED");
    record.setBorrowDate(LocalDate.now());
    record.setDueDate(LocalDate.now().plusDays(daysToAdd: 1));

    borrowRecordRepository.save(record);
}
```

Controller

Delete Borrow

```
@PostMapping("/borrow/delete/{id}")
public String deleteBorrow(@PathVariable("id") Long id, RedirectAttributes redirectAttributes) {
    try {
        libraryService.deleteBorrowRecord(id);
        redirectAttributes.addFlashAttribute(attributeName: "successMessage", attributeValue: "Borrow record deleted.");
    } catch (Exception e) {
        redirectAttributes.addFlashAttribute(attributeName: "errorMessage", e.getMessage());
    }
    return "redirect:/librarian/process_return";
}

@PostMapping("/user/borrowRecord/{id}/delete")
public String deleteBorrowRecord(@PathVariable("id") Long recordId,
    RedirectAttributes redirectAttributes) {
    try {
        libraryService.deleteBorrowRecord(recordId);
        redirectAttributes.addFlashAttribute(attributeName: "successMessage", attributeValue: "Borrow record deleted.");
    } catch (Exception e) {
        redirectAttributes.addFlashAttribute(attributeName: "errorMessage", e.getMessage());
    }
    return "redirect:/user/borrowRecord";
}
```

libraryController

borrowRecordController

Service

```
@Transactional
public void deleteBorrowRecord(Long recordId) {
    BorrowRecord record = borrowRecordRepository.findById(recordId)
        .orElseThrow(() -> new RuntimeException(message: "Borrow record not found"));

    Book book = record.getBook();
    if (book != null) {
        // Return quantity if record was not returned
        Integer q = book.getQuantity();
        book.setQuantity((q == null ? 0 : q) + 1);
        book.setAvailable(book.getQuantity() != null && book.getQuantity() > 0);
        bookRepository.save(book);
    }

    borrowRecordRepository.delete(record);
}
```

Controller

Return Borrow

```
@PostMapping("/borrow/return/{id}")
public String confirmReturn(@PathVariable("id") Long id, RedirectAttributes redirect) {
    libraryService.confirmReturn(id);
    redirect.addFlashAttribute(attributeName: "successMessage", attributeValue: "Return confirmed successfully");
    return "redirect:/librarian/process_return";
}
```

libraryController

Service

```
@Transactional
public void confirmReturn(Long recordId) {
    BorrowRecord record = borrowRecordRepository.findById(recordId)
        .orElseThrow(() -> new RuntimeException(message: "Borrow record not found"));

    if (!"BORROWED".equals(record.getStatus())) {
        throw new RuntimeException(message: "Only borrowed books can be returned");
    }

    LocalDate today = LocalDate.now();
    record.setReturnDate(today);

    if (today.isAfter(record.getDueDate())) {
        record.setStatus(status: "RETURNED_LATE");
    } else {
        record.setStatus(status: "RETURNED_ON_TIME");
    }

    Book book = record.getBook();
    if (book != null) {
        Integer q = book.getQuantity() == null ? 0 : book.getQuantity();
        book.setQuantity(q + 1);
        book.setAvailable(book.getQuantity() > 0);
        bookRepository.save(book);
    }

    borrowRecordRepository.save(record);

    if ("RETURNED_LATE".equals(record.getStatus())) {
        User user = record.getUser();
        long lateCount = borrowRecordRepository.countByUserIdAndStatus(user.getId(), status: "RETURNED_LATE");

        if (lateCount >= 3 && user.getActive()) {
            user.setActive(active: false);
            userRepository.save(user);
        }
    }
}
```

Repository

```
long countByUserIdAndBorrowDateAfter(Long userId, LocalDate date);

List<BorrowRecord> findByUserIdAndStatusInOrderByBorrowDateDesc(Long userId, List<String> statuses);

long countByUserIdAndStatus(Long userId, String status);

long countByUserIdAndStatusAndDueDateBefore(Long userId, String status, LocalDate date);

long countByUserIdAndStatusAndDueDateBetween(Long userId, String status, LocalDate start, LocalDate end);

long countByUserIdAndStatusInAndCreateddtBetween(Long userId, List<String> statuses, LocalDateTime start, LocalDateTime end);

BorrowRecord findTopByUserIdOrderByBorrowDateDesc(Long userId);

long countByStatus(String status);
long countByStatusAndDueDateBefore(String status, LocalDate date);
List<BorrowRecord> findByUserIdOrderdyCreateddtDesc(Long userId);

// Use @Param for named parameter
@Query("SELECT br FROM BorrowRecord br WHERE br.status = :status")
List<BorrowRecord> findByStatus(@Param("status") String status);
List<BorrowRecord> findByUserIdAndStatusInOrderByCreateddtDesc(
    Long userId,
    List<String> statuses
);

List<BorrowRecord> findAllByOrderByCreateddtDesc();

@Query("SELECT br FROM BorrowRecord br
JOIN br.book b
WHERE
    LOWER(b.title) LIKE LOWER(CONCAT('%', :query, '%'))
    OR LOWER(b.isbn) LIKE LOWER(CONCAT('%', :query, '%'))
    OR LOWER(b.author) LIKE LOWER(CONCAT('%', :query, '%'))
    OR LOWER(b.email) LIKE LOWER(CONCAT('%', :query, '%'))
    OR LOWER(u.email) LIKE LOWER(CONCAT('%', :query, '%'))
    OR LOWER(u.fullName) LIKE LOWER(CONCAT('%', :query, '%'))
")
List<BorrowRecord> searchBorrowRecords(@Param("query") String query);
}
```

Controller

View Record

```
@GetMapping("/records")
public String manageBorrows(Model model) {
    List<BorrowRecord> pendingRecords = libraryService.getAllPendingBorrowRecords();
    model.addAttribute(attributeName: "borrowedItems", pendingRecords);
    return "librarian/manage-borrows";
}

@GetMapping("/process_return")
public String processReturnPage(Model model) {
    List<BorrowRecord> borrowedItems =
        libraryService.getAllBorrowRecordsForLibrarian();

    model.addAttribute(attributeName: "borrowedItems", borrowedItems);
    model.addAttribute(attributeName: "activeTab", attributeValue: "process_return");

    return "librarian/process_return";
}

@GetMapping("/user/{id}/history")
public String viewUserHistory(@PathVariable("id") Long id, Model model, RedirectAttributes redirect) {
    return libraryService.findUserById(id)
        .map(user -> {
            List<BorrowRecord> history = libraryService.getUserHistory(user.getId());
            model.addAttribute(attributeName: "user", user);
            model.addAttribute(attributeName: "history", history);
            return "librarian/user_history";
        })
        .orElseGet(() -> {
            redirect.addFlashAttribute(attributeName: "error", attributeValue: "User not found");
            return "redirect:/librarian/manage_user";
        });
}
```

libraryController

```
@GetMapping("/user/borrowRecord")
public String borrowRecord(Model model, Principal principal) {
    model.addAttribute(attributeName: "borrowRecord", libraryService.getAllBorrowRecords(principal.getName()));
    return "user/borrowRecord";
}

@GetMapping("/records/search")
public String searchBorrowRecords(
    @RequestParam("query") String query,
    Model model) {

    List<BorrowRecord> results =
        libraryService.searchBorrowRecords(query);

    model.addAttribute(attributeName: "borrowedItems", results);
    model.addAttribute(attributeName: "activeTab", attributeValue: "process_return");
    model.addAttribute(attributeName: "query", query);

    return "librarian/process_return";
}
```

borrowRecocordController

Service

```
public List<BorrowRecord> getAllPendingBorrowRecords() {
    return borrowRecordRepository.findByStatus(status: "BOOKED");
}

@Transactional(readOnly = true)
public List<BorrowRecord> getAllBorrowRecordsForLibrarian() {
    return borrowRecordRepository.findAllByOrderByCreatedAtDesc();
}

public List<BorrowRecord> getUserHistory(Long userId) {
    return borrowRecordRepository.findByUserIdOrderdyCreatedAtDesc(userId);
}

public List<BorrowRecord> getAllBorrowRecords(String email) {
    User user = userRepository.findById(email)
        .orElseThrow(() -> new RuntimeException(message: "User not found"));
    return borrowRecordRepository.findByUserIdOrderdyCreatedAtDesc(user.getId());
}

@Transactional(readOnly = true)
public List<BorrowRecord> searchBorrowRecords(String query) {
    if (query == null || query.trim().isEmpty()) {
        return borrowRecordRepository.findAllByOrderByCreatedAtDesc();
    }
    return borrowRecordRepository.searchBorrowRecords(query.trim());
}
```

Borrow Management

Book Reservation

- **getActiveBorrowCount(Long userId){}**
 - count BOOKED and BORROWED
- **bookReservation(Long bookId, String email){}**
 - check book available
 - limit maximum 3 book
 - create record : status="BOOKED"
 - book quantity : -1

Return Borrow

- **returnBook(Long recordId){}**
 - Compare the date return with due date
 - Before/On date: "RETURNED_ON_TIME"
 - After: "RETURNED_LATE"
 - 3 times automatic set user to deactivate
 - book quantity : +1

Confirms Borrow

- **confirmBorrow(Long recordId){}**
 - change status "BOOKED" → "BORROWED"
 - borrow date set now
 - due date auto-calculated (today+14)

Delete Borrow

- **deleteBorrowRecord(Long recordId){}**
 - delete record
 - book quantity: +1

View Borrow

- *For Librarian*
 - **getAllPendingBorrowRecords(){}**
 - view all pending borrow record
 - **getAllBorrowRecordsForLibrarian() {}**
 - view all borrow record
 - **getUserHistory(Long userId)**
 - View all borrow record in user history
 - **searchBorrowRecords(String query)**
 - view by search
- For User
 - **getAllBorrowRecords(String email)**
 - view their record

TECHNOLOGY USE



Spring Boot



Spring Data JPA



MySql



Spring MVC



Thymeleaf



Tailwind CSS



Spring Security



Java (JDK 21+)



HTML5

STRENGTHS

- Efficient Book Management.
- Borrowing Limit Control.
- Easy Monitoring of Borrowed Books.
- User Authentication and Security.
- Scalable System Design.



WEAKNESS

- Limited Borrowing Features.
- Dependency on Internet and Server Availability.
- No Notification System



CHALLENGE

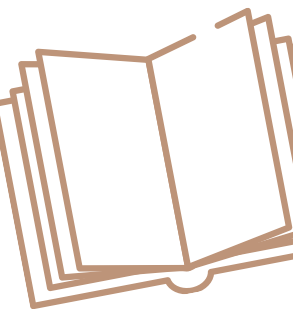
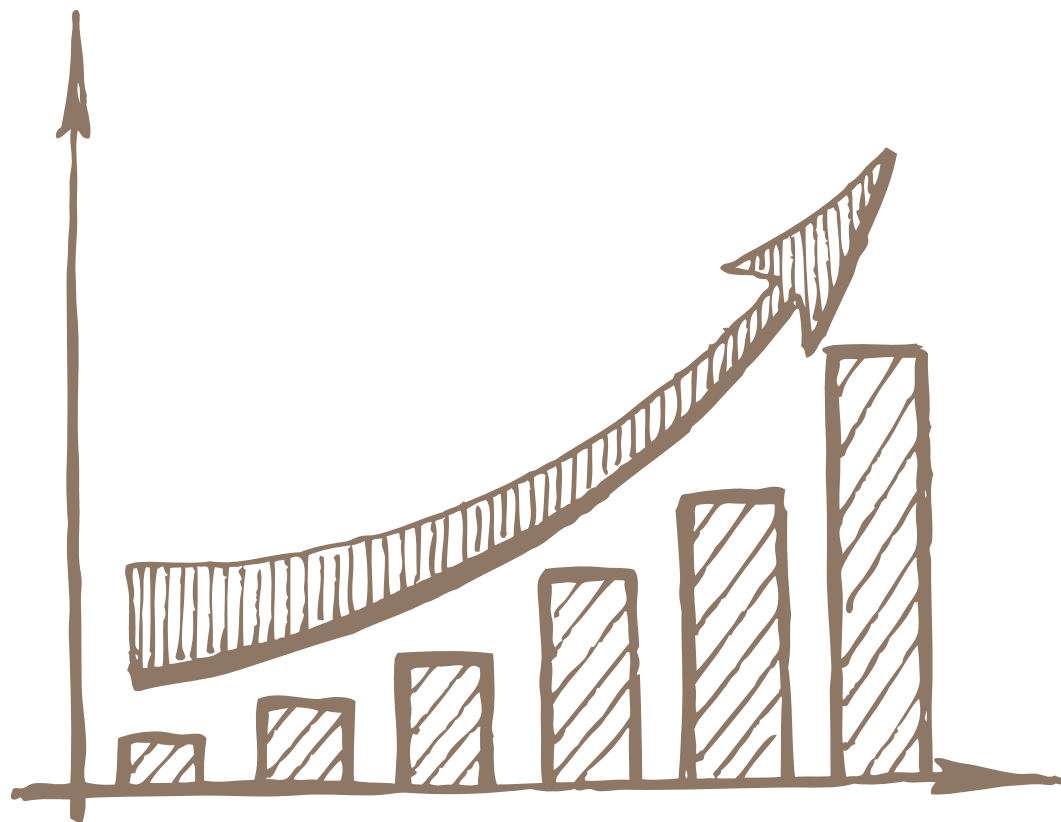
- Managing time and debugging errors
- Ensuring that a user can borrow only three books at one time.
- Preventing multiple users from borrowing the same book simultaneously.
- Ensuring secure user access.
- Restricting borrowing actions for user to ensure only logged-in users can borrow books.



CONCLUSION

The Library Borrowing and Book Tracking System was designed to improve the management of library operations by automating the borrowing and tracking of books. The system helps librarians and users manage book records efficiently while reducing the need for manual paperwork.

One of the key features of the system is the limitation that allows a user to borrow a maximum of three books at a time.



DEMO





THANK YOU

